WIKI BUILDER

WIKI BUILDER

1) Contexte & périmètre

Le projet consiste à développer une **API REST** avec **Laravel** permettant de gérer un catalogue **multi-jeux** (équipements/objets/armes, accessoires/modificateurs, tags/traits) et un module **Builder** pour assembler et comparer des configurations (“builds/loadouts”).

L’API expose des endpoints permettant de **lister, filtrer et trier** les contenus d’un jeu sélectionné (toutes les données sont rattachées à un jeu ; un build ne peut contenir que des éléments appartenant au même jeu), ainsi que de **créer, afficher, modifier, supprimer** des builds via une agrégation simple de statistiques stockées (pas de simulation).

Périmètre inclus :

- ➢ Gestion d’un catalogue de jeux.
- ➢ Gestion de profils de build par jeu (optionnel : spécialiste COD, classe RPG, etc.).
- ➢ Gestion d’équipements par jeu (type/slot, rareté, tier/level, score, stats simples).
- ➢ Gestion d’accessoires/modificateurs par jeu (ex : attachments) + gestion des compatibilités équipement↔accessoire (many-to-many).
- ➢ Gestion de tags/traits associés aux équipements (many-to-many) pour faciliter recherche/filtrage.
- ➢ Gestion de builds : profil optionnel + sélection d’équipements + accessoires sélectionnés et comparaison simple (agrégats).
- ➢ Sécurisation : **API Key** stockée en base + contrôle de rôle (**RBAC minimal**) pour protéger les routes d’écriture.

Hors périmètre :

- ➢ Simulation avancée (DPS/TTK réels, perks détaillés, synergies complexes).
- ➢ Scraping/import automatique de données.

2) Cibles et utilisateurs

- **Gamer** : Consulte le catalogue, filtre et compare des builds.
- **Prof / évaluateur** : Lance l'API via Docker et teste les endpoints via Postman/cURL.
- **Administrateur** : Gère le référentiel via routes protégées (API Key + rôle).

3) Fonctionnalités

Catalogue :

- Lister/consulter les jeux.
- Lister/consulter les profils de build d'un jeu (optionnel).
- Lister/consulter les équipements d'un jeu avec filtres (slot, rareté, tier/level, score min/max, tag) + tri (nom, rareté, score) + pagination.
- Lister/consulter les tags et les équipements associés.
- Lister/consulter les accessoires/modificateurs d'un jeu (par type) et les équipements compatibles.

Builder :

- CRUD build : titre, profil optionnel, liste d'équipements, accessoires sélectionnés.
- Endpoint résumé : agrégation simple (ex : somme/moyenne) (total_attack/defense/utility/score).
- Endpoint comparaison : diff des agrégats entre deux builds.

Règles métier (validées côté API) :

- Un build est mono-jeu : les équipements/modificateurs/profil doivent appartenir au même jeu.
- Un modificateur ne peut être ajouté à un équipement que si la compatibilité existe (table de compatibilité).
- Résumé : somme stats équipements + somme deltas modificateurs.

4) Partie technique

- **Laravel** : Models, Migrations, Controllers, Routes.
- **MySQL** : schéma géré via migrations + seed dataset.
- **Resources** (JSON structuré) + **Form Requests** (validation/autorisation).
- **Middlewares : Throttle** + **CheckApiKey** (clé en base) + **CheckRole** (role sur user).
- **Docker Compose** : `app` + `db`, README de démarrage reproductible.

5) MCD (Modèle conceptuel de donnée)

(relations nommée, pas de FK, pas de pivot)

Entité:

- **GAME**: game_id, name, slug, genre
- **BUILD_PROFILE** *(optional per game)*: build_profile_id, name, description
- **EQUIPMENT_ITEM**: equipment_item_id, name, slot, rarity, tier_level, score, attack, defense, utility, accuracy, mobility
- **EQUIPMENT_MODIFIER** *(attachment/modifier)*: equipment_modifier_id, name, modifier_type, description, delta_score, delta_attack, delta_defense, delta_utility, delta_accuracy, delta_mobility
- **TAG**: tag_id, label
- **BUILD**: build_id, title, description, created_at
- **API_KEY**: api_key_id, key, label
- **USER**: user_id, username, email, role

Relations nommée + cardinalités:

- **OWNS_BUILD_PROFILES**: GAME (1,1) —— (0,N) BUILD_PROFILE
- **OWNS_EQUIPMENT_ITEMS**: GAME (1,1) —— (0,N) EQUIPMENT_ITEM
- **OWNS_EQUIPMENT_MODIFIERS**: GAME (1,1) —— (0,N) EQUIPMENT_MODIFIER
- **OWNS_BUILDS**: GAME (1,1) —— (0,N) BUILD
- **USES_BUILD_PROFILE** *(optional)*: BUILD_PROFILE (0,N) —— (0,1) BUILD
- **TAGGED_WITH** *(many-to-many)*: EQUIPMENT_ITEM (0,N) —— (0,N) TAG
- **COMPATIBLE_WITH** *(many-to-many)*: EQUIPMENT_ITEM (0,N) —— (0,N) EQUIPMENT_MODIFIER
- **COMPOSED_OF**: BUILD (1,N) —— (0,N) EQUIPMENT_ITEM
- **USES_MODIFIERS** *(many-to-many)*: BUILD (0,N) —— (0,N) EQUIPMENT_MODIFIER
- **CREATED_BY** *(optional)*: USER (0,N) —— (0,1) BUILD

Notes:

- Les tags sont globaux (réutilisables entre tous les jeux).
- Règle métier : un build ne peut référencer que des items/modificateurs/profils du même jeu (validation côté API).

6) MPD (Modèle physique de donnée)

(table MySQL, pivots, types, FK)

api_keys

- id BIGINT UNSIGNED PK AI
- key VARCHAR(64) NOT NULL UNIQUE
- label VARCHAR(100) NULL
- is_active TINYINT(1) NOT NULL DEFAULT 1
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL

users

- id BIGINT UNSIGNED PK AI
- username VARCHAR(80) NOT NULL UNIQUE
- email VARCHAR(190) NOT NULL UNIQUE
- role VARCHAR(30) NOT NULL
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL

games

- id BIGINT UNSIGNED PK AI
- name VARCHAR(120) NOT NULL
- slug VARCHAR(120) NOT NULL UNIQUE
- genre VARCHAR(80) NULL
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL

build_profiles

- id BIGINT UNSIGNED PK AI
- game_id BIGINT UNSIGNED NOT NULL FK → games.id
- name VARCHAR(120) NOT NULL
- description TEXT NULL
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL
- UNIQUE (game_id, name)
- INDEX (game_id)

equipment_items

- id BIGINT UNSIGNED PK AI
- game_id BIGINT UNSIGNED NOT NULL FK → games.id
- name VARCHAR(140) NOT NULL
- slot VARCHAR(60) NOT NULL
- rarity VARCHAR(40) NOT NULL
- tier_level VARCHAR(20) NULL
- score INT NOT NULL DEFAULT 0
- attack INT NULL
- defense INT NULL
- utility INT NULL
- accuracy INT NULL
- mobility INT NULL
- metadata JSON NULL
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL
- UNIQUE (game_id, name)
- INDEX (game_id), INDEX (slot), INDEX (rarity), INDEX (game_id, score)

equipment_modifiers

- id BIGINT UNSIGNED PK AI
- game_id BIGINT UNSIGNED NOT NULL FK → games.id
- name VARCHAR(140) NOT NULL
- modifier_type VARCHAR(60) NOT NULL
- description TEXT NULL
- delta_score INT NOT NULL DEFAULT 0
- delta_attack INT NULL
- delta_defense INT NULL
- delta_utility INT NULL
- delta_accuracy INT NULL
- delta_mobility INT NULL
- metadata JSON NULL
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL
- UNIQUE (game_id, name)
- INDEX (game_id), INDEX (modifier_type), INDEX (game_id, modifier_type)

tags

- id BIGINT UNSIGNED PK AI
- label VARCHAR(80) NOT NULL UNIQUE
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL

builds

- id BIGINT UNSIGNED PK AI
- game_id BIGINT UNSIGNED NOT NULL FK → games.id
- user_id BIGINT UNSIGNED NULL FK → users.id
- build_profile_id BIGINT UNSIGNED NULL FK → build_profiles.id
- title VARCHAR(140) NOT NULL
- description TEXT NULL
- created_at TIMESTAMP NULL
- updated_at TIMESTAMP NULL
- INDEX (game_id), INDEX (user_id), INDEX (build_profile_id)

equipment_item_tags (pivot)

- equipment_item_id BIGINT UNSIGNED NOT NULL FK → equipment_items.id ON DELETE CASCADE
- tag_id BIGINT UNSIGNED NOT NULL FK → tags.id ON DELETE CASCADE
- PRIMARY KEY (equipment_item_id, tag_id)
- INDEX (tag_id)

equipment_item_compatible_modifiers (pivot)

- equipment_item_id BIGINT UNSIGNED NOT NULL FK → equipment_items.id ON DELETE CASCADE
- equipment_modifier_id BIGINT UNSIGNED NOT NULL FK → equipment_modifiers.id ON DELETE CASCADE
- PRIMARY KEY (equipment_item_id, equipment_modifier_id)
- INDEX (equipment_modifier_id)

build_equipment_items (pivot)

- build_id BIGINT UNSIGNED NOT NULL FK → builds.id ON DELETE CASCADE
- equipment_item_id BIGINT UNSIGNED NOT NULL FK → equipment_items.id ON DELETE CASCADE
- position SMALLINT UNSIGNED NULL
- PRIMARY KEY (build_id, equipment_item_id)
- INDEX (equipment_item_id)

build_equipment_modifiers (pivot)

- build_id BIGINT UNSIGNED NOT NULL FK → builds.id ON DELETE CASCADE
- equipment_item_id BIGINT UNSIGNED NOT NULL FK → equipment_items.id ON DELETE CASCADE
- equipment_modifier_id BIGINT UNSIGNED NOT NULL FK → equipment_modifiers.id ON DELETE CASCADE
- PRIMARY KEY (build_id, equipment_item_id, equipment_modifier_id)
- INDEX (equipment_modifier_id), INDEX (equipment_item_id)

Contraintes métier (API) :

- ➢ Un build est mono-jeu : les items/modificateurs/profils référencés doivent correspondre à build.game_id.
- ➢ Le modificateur doit être compatible avec l'item via la table equipment_item_compatible_modifiers.
- ➢ Le middleware API Key valide api_keys.key en base ; le middleware rôle utilise users.role.

AI_INSTRUCTIONS

AI_INSTRUCTIONS.md

AI_INSTRUCTIONS.md — Wiki Builder (Laravel API)

Goal

Generate a Laravel REST API for the Wiki Builder project using the provided MPD/DBML and business rules.

Non-negotiable business rules

1. **Single-game build rule** A build can only reference `build_profile`, `equipment_items` and `equipment_modifiers` with the same `game_id` as the build.

2. **Compatibility rule** A modifier can be added to an item only if `(equipment_item_id, equipment_modifier_id)` exists in `equipment_item_compatible_modifiers`.

3. **Summary formula (NULL = 0)** Build summary must compute totals treating NULL as 0:

    - `total_score = SUM(equipment_items.score) + SUM(equipment_modifiers.delta_score)`
    - Same logic for `attack / defense / utility / accuracy / mobility` using `delta_*`.
4. **Security (course-oriented)**

    - All write routes require header `API-KEY` validated against `api_keys.key` (and `is_active = true`) in DB.
    - RBAC minimal: admin-only routes require a user with `users.role == "admin"`.
    - **User identification is simplified**: requests include header `X-USER-ID`. The `CheckRole` middleware loads the user from DB using `X-USER-ID` and checks `role`.
    - Throttle: 60 requests/minute on all routes.

API endpoints (minimum)

Public (read-only)

GET /games
GET /games/{id}
GET /games/{id}/build-profiles
GET /games/{id}/equipment-items
GET /games/{id}/equipment-modifiers
GET /tags
GET /equipment-items/{id}
GET /equipment-modifiers/{id}
GET /builds/{id}

Write (protected by `API-KEY`)

POST /builds
PUT /builds/{id}
DELETE /builds/{id}

POST /build-profiles
PUT /build-profiles/{id}
DELETE /build-profiles/{id}

POST /equipment-items
PUT /equipment-items/{id}
DELETE /equipment-items/{id}

POST /equipment-modifiers
PUT /equipment-modifiers/{id}
DELETE /equipment-modifiers/{id}

POST
/equipment-items/{equipment_item_id}/compatible-modifiers/{equipment_modifier_id}
DELETE
/equipment-items/{equipment_item_id}/compatible-modifiers/{equipment_modifier_id}

POST /equipment-items/{equipment_item_id}/tags/{tag_id}
DELETE /equipment-items/{equipment_item_id}/tags/{tag_id}

Builder calculations

GET /builds/{id}/summary
POST /builds/compare

Request / Response contracts

`GET /builds/{id}/summary`


```
{
  "build_id": 1,
  "items_count": 0,
  "modifiers_count": 0,
  "totals": {
    "score": 0,
    "attack": 0,
    "defense": 0,
    "utility": 0,
    "accuracy": 0,
    "mobility": 0
  }
}
```


`POST /builds/compare`

Request body:


```
{
  "left_build_id": 1,
  "right_build_id": 2
}
```


Response body:


```
{
  "left": {
    "build_id": 1,
    "totals": { "score": 0, "attack": 0, "defense": 0, "utility": 0, "accuracy": 0, "mobility": 0 }
  },
  "right": {
    "build_id": 2,
    "totals": { "score": 0, "attack": 0, "defense": 0, "utility": 0, "accuracy": 0, "mobility": 0 }
  },
  "diff": {
    "score": 0, "attack": 0, "defense": 0, "utility": 0, "accuracy": 0, "mobility": 0
  }
}
```

What to generate (order matters)

Phase 1 — Database & boot

- Laravel migrations for all tables in the MPD (including pivots, composite PKs, indexes).
- Seeders:
  - 2 games
  - 5 build_profiles
  - 20 equipment_items
  - 20 equipment_modifiers
  - compatibility rows
  - tags + item tags
  - 1 admin user (`role=admin`) + 1 standard user (`role=user`)
  - 1 api_key for Postman testing (`is_active=true`)
- `docker-compose.yml` (app + db) + `.env.example`

Phase 2 — Models (Eloquent)

Generate models with relationships:

- `Game` → `hasMany` BuildProfile, EquipmentItem, EquipmentModifier, Build
- `EquipmentItem` → `belongsTo` Game
- `EquipmentItem` → `belongsToMany` Tag (pivot: `equipment_item_tags`)
- `EquipmentItem` → `belongsToMany` EquipmentModifier (pivot: `equipment_item_compatible_modifiers`)
- `Build` → `belongsTo` Game; `belongsTo` optional BuildProfile; `belongsTo` optional User
- `Build` → `belongsToMany` EquipmentItem (pivot: `build_equipment_items`, with `position`)
- `Build` → `belongsToMany` EquipmentModifier (pivot: `build_equipment_modifiers`, with required `equipment_item_id`)

Phase 3 — API structure

- Controllers (CRUD) for: `games`, `build_profiles`, `equipment_items`, `equipment_modifiers`, `tags`, `builds`
- Filtering / sorting / pagination on `equipment_items` and `equipment_modifiers` by `game_id` and fields: `slot`, `rarity`, `score` range, `tag`, `modifier_type`

Phase 4 — Validation & responses

- Form Requests for all create/update endpoints and for the compare endpoint.

- Enforce business rules in requests or service layer.
- Use API Resources for consistent JSON output.
- Provide `ApiResponse` helper (success/error) and consistent validation error format.

Phase 5 — Middlewares

- `CheckApiKey`: reads `API-KEY` header, validates in DB, returns `401` if invalid or inactive.
- `CheckRole`: reads `X-USER-ID` header, loads user from DB, checks `role`, returns `403` if forbidden.
- Register middleware aliases in Laravel (`Kernel.php` or `bootstrap/app.php` depending on Laravel version).

Output rules

- Generate complete files (no snippets).
- Keep naming consistent with the MPD.
- English comments only.
- Provide a `README.md` with exact commands:

docker compose up -d --build
php artisan migrate --force
php artisan db:seed --force

- Include example `curl` calls using `API-KEY` and `X-USER-ID` headers.